

PYTHON OOP & FUNDAMENTALS

Comprehensive 30-Question Revision Blueprint (Days 1–9)

Scope: Syntax, Variables, Operators, Strings, Collections (Lists, Tuples, Sets, Dictionaries), User Input, Conditionals, and Object-Oriented Programming (Constructors, Instance Attributes/Methods, Class Variables, Class Methods, and Static Methods).

Constraints: Strictly NO loops, custom global functions, external libraries, file handling, or inheritance. Focus is entirely on structured algorithmic logic combined with core OOP architecture.

Note: This document contains the question sets only. Solutions are withheld for self-assessment.

Section 1: Core OOP, Logic & Data Structures (Questions 1–10)

Question 1 – Employee Information System

Create a class named `Employee`. The constructor should accept `first_name`, `last_name`, `age`, `department`, and `salary`. It must automatically generate a `full_name` attribute and a `email` attribute formatted as `firstname.lastname@company.com` (all lowercase).

- Add a class variable `company_name` initialized to "TechNova Ltd".
- Add a class variable `raise_amount` set to 1.10.
- Create a regular method `display_information()` that prints all details in a clean, professional layout.
- Create a regular method `increase_salary()` that updates the salary attribute using `raise_amount`.
- Create a class method `change_company(new_name)` that updates the company name for all instances.
- Create a static method `is_adult(age)` that returns `True` if age is 18 or above, and `False` otherwise.
- Instantiate three unique employee objects. Display their profiles, execute a raise, update the corporate name via the class method, and print the updated records to prove the shared class attribute functions correctly.

Question 2 – Student Admission Evaluation

Create a class called `Student` whose constructor receives `student_name`, `math_marks`, `english_marks`, and `physics_marks`. The constructor must automatically calculate `total_marks` and `average_marks`.

- Create a regular method named `evaluate()` that assigns an admission status based on the average mark according to this matrix:

Average Mark Range	Admission Status
90 – 100	Scholarship Granted
80 – 89	Direct Admission
70 – 79	Admission Pending Interview
60 – 69	Waiting List
Below 60	Rejected

- The system should display the Student's Name, Total Marks, Average Mark, and final Admission Status.
Tricky part: Ensure no external calculation is needed; instantiation handles the math.

Question 3 – Banking Account Transaction Safety

Design a `BankAccount` class. The constructor parameters should take `account_holder`, `account_number`, and an `initial_balance`.

- Implement three regular methods: `deposit(amount)`, `withdraw(amount)`, and `display_balance()`.
- Enforce rigorous execution safety rules using conditional logic:
 - A withdrawal cannot exceed the current balance. If attempted, print a clear denial message.
 - Negative or zero values are strictly banned for both deposits and withdrawals.
- Simulate a series of safe and unsafe operations, printing explicit operational feedback status line-by-line.

Question 4 – Electronics Shop Profit Analyzer

Create a class named `Product` that requires a `product_name`, `buying_price`, and `selling_price` during initialization. The constructor must instantly compute the absolute profit and the `profit_percentage`.

- Write a method that prints the financial metrics neatly.
- Using logical conditionals, categorize the commercial outcome into one of three tiers: "Huge Profit" (if profit percentage is $\geq 40\%$), "Small Profit" (if greater than 0% but below 40%), or "Loss" (if zero or negative).

Question 5 – Smart Password Checker

Write a script that prompts a user to enter a proposed password via `input()`. Without using loops, validate the structural integrity of the string using comprehensive string logic and comparison operators.

- Check for these conditions: must be at least 8 characters long, must contain at least one uppercase letter, must contain at least one lowercase letter, must contain a number, and must not contain any blank spaces.
- If any validation fails, print the specific reason why it failed (isolate and output the first error caught). If all parameters match, print "Password Accepted".

Question 6 – Vehicle Registration System

Create a class called `Vehicle` with attributes `brand`, `model`, `year`, and `price`. Have the constructor determine the current age of the vehicle dynamically based on the current year (2026).

- Create instance methods to classify the asset tier: "Brand New" (≤ 1 year old), "Fairly Used" (2 to 7 years old), or "Old" (> 7 years old).
- Add a class variable `dealer_name` and a class method to modify it.
- Incorporate a static method `is_classic(year)` that returns `True` if a car's age is greater than 25 years.

Question 7 – Electricity Bill Calculator

Build an independent script that captures a customer's name and total units consumed. Calculate the bill using a tiered pricing structure: First 50 units at KSh 12/unit; next 100 units at KSh 15/unit; any units above 150 at KSh 18/unit.

- Categorize the household consumption profile based on total monetary cost: "Low Consumption" (Below KSh 1,000), "Moderate Consumption" (KSh 1,000–3,000), or "High Consumption" (Above KSh 3,000). Print a clean utility breakdown statement.

Question 8 – Hospital Patient Record Diagnostics

Create a `Patient` class tracking name, age, weight, and temperature (Celsius).

- Design clinical evaluation methods: Temperature check (Below 36.0 $\rightarrow$ "Low Temperature"; 36.0 to 37.5 $\rightarrow$ "Normal"; Above 37.5 $\rightarrow$ "Fever"). Age check (Below 18 $\rightarrow$ "Minor"; 18–59 $\rightarrow$ "Adult"; 60+ $\rightarrow$ "Senior Citizen").
- Output the structured diagnostic chart showing patient telemetry and categorization.

Question 9 – University Registration Ledger

Create a class named `UniversityStudent` capturing `first_name`, `last_name`, `course`, `year`, and `fee_balance`. Automatically generate a corporate student email: `firstname.lastname@university.ac.ke`.

- Implement methods to return fee clearance status: 0 → "Cleared"; 1–30,000 → "Small Balance"; 30,001–70,000 → "Moderate Balance"; Above 70,000 → "Large Balance".
- Add a static method to evaluate if a student is eligible for graduation based solely on whether their fee balance is zero.

Question 10 – Employee Performance Evaluation

Create an advanced `Employee` evaluation class with attributes: `name`, `department`, `salary`, and a `performance_score` (scalar 0–100).

- Develop an internal rating method: 90–100 → "Outstanding"; 80–89 → "Excellent"; 70–79 → "Very Good"; 60–69 → "Good"; 50–59 → "Average"; Below 50 → "Needs Improvement".
- Develop a bonus allocation method that modifies salary or computes a payout: Outstanding gets a 20% bonus; Excellent gets 15%; Very Good gets 10%; Good gets 5%; others receive 0%. Print a comprehensive compensation adjustment report.

Section 2: Intermediate Architectural Scenarios (Questions 11–20)

Question 11 – Library Management System

Create a class called `Book` containing `title`, `author`, `isbn_number`, `price`, and `number_of_copies`. The constructor must compute the total value of stock held for that book title.

- Include a class variable `library_name`.
- Create an instance method `availability()` returning: "Out of Stock" if copies are 0; "Low Stock" if 1 to 5; and "Available" if greater than 5.
- Add a static method `is_expensive(price)` returning `True` if the cost exceeds KSh 5,000.

Question 12 – Mobile Phone Inventory Control

Design an inventory class named `Phone` with `brand`, `model`, `storage_gb`, `ram_gb`, and `price`. Concatenate parameters to form an attribute called `full_name`.

- Implement checking logic to categorize hardware capacity based on storage volume: Less than 64GB → "Entry Storage"; 64GB–127GB → "Standard"; 128GB–255GB → "High-Capacity"; 256GB+ → "Enterprise Tier".
- Categorize pricing strategy: Budget (Below KSh 20,000), Mid-range (KSh 20,000–50,000), Premium (Above KSh 50,000).

Question 13 – Electricity Meter Registration Tracker

Create an `ElectricityCustomer` class processing `customer_name`, `meter_number`, and `units_consumed`. Compute billing progressively within the constructor using tiered boundaries (0-100 units at KSh 15, next 100 units at KSh 18, and units over 200 at KSh 22).

- Determine customer value bands (Bronze, Silver, Gold, Platinum) depending on the magnitude of their final bill statement.

Question 14 – Hotel Reservation Billing System

Design a `Guest` tracker class for a resort that accepts `name`, `number_of_nights`, and `cost_per_night`. Auto-calculate total transaction value.

- Create classification methods for room status based on nightly costs: Economy, Standard, Deluxe, or Executive.
- Classify length of stay profiles: Short Stay (1–3 nights), Medium Stay (4–7 nights), or Long Stay (8+ nights).

Question 15 – Academic Exam Result Analyzer

Design a Candidate scorecard class accepting a student's name along with unique percentage marks for Mathematics, English, Chemistry, and Physics. Auto-calculate total and average metrics.

- Output standard letter grades: Average $\geq 90 \rightarrow A$; 80–89 $\rightarrow B$; 70–79 $\rightarrow C$; 60–69 $\rightarrow D$; Below 60 $\rightarrow E$.
- Use explicit conditional strings to append career pathway suggestions based on metrics (e.g., if Average ≥ 80 and Physics ≥ 75 , append "Eligible for Engineering").

Question 16 – Car Dealership Valuations

Create a Car tracking model parsing brand, model, manufacturing_year, buying_price, and selling_price. Calculate raw profit margin values inside the initializer.

- Categorize life-cycle metrics: Brand New, Modern, Used, or Classic.
- Map financial yield categories: Excellent, Good, Fair, or Loss based on the percentage profit clearing thresholds.

Question 17 – Airline Ticket Booking Manifest

Design a Passenger booking class handling passenger_name, destination, ticket_price, and luggage_weight (kg).

- Write regular methods to evaluate cabin class routing by ticket cost thresholds (Economy, Premium Economy, Business, First Class).
- Incorporate a static method that checks whether luggage weight breaches international limits (exceeding 23kg), returning a flat flag.

Question 18 – Online Shopping POS Checkout

Design a retail checkout module using a ShoppingCart entity that accepts a customer_name, product_name, quantity, and unit_price.

- Calculate gross value. Apply a scaling conditional volume discount framework: Base gross above KSh 20,000 gets 15%; above KSh 10,000 gets 10%; above KSh 5,000 gets 5%; otherwise 0%. Output the precise original amount, discount savings applied, and final payable invoice amount.

Question 19 – Clinical Invoice Billing System

Create a medical Patient invoice tracking entity mapping name, consultation_fee, laboratory_fee, and medication_fee. Auto-sum total expenditures.

- Isolate tiers of case management: Low Cost, Moderate Cost, High Cost, or Critical Cost.

- Add a method that evaluates if a patient qualifies for a safety-net financial waiver program based on total costs exceeding KSh 50,000.

Question 20 – Corporate Payroll Deduction Engine

Create an Employee payroll processing engine that accepts `first_name`, `last_name`, `basic_salary`, `overtime_pay`, and `allowances`.

- Calculate `gross_salary`. Automatically calculate standard statutory deductions: NHIF (2.5% of gross), NSSF (6% of gross), and Housing Levy (1.5% of gross). Calculate total deductions and net final cash salary.
- Map employee ranks based on net salary output: Below 30k → Entry; 30k–60k → Mid; 60k–100k → Senior; 100k+ → Executive. Show a complete payroll slip layout.

Section 3: High-Complexity & Expert Scenarios (Questions 21–30)

Question 21 – University Hostel Allocation Matrix

Design a student allocation class called `HostelStudent` accepting `first_name`, `last_name`, `gender`, `age`, `year_of_study`, and `fee_balance`.

- Establish a class variable `hostel_name`. Implement a regular method `hostel_eligibility()` that checks clearance rules using complex logical compound terms: fee balance must be exactly 0, age must be ≥ 18 , and priority is granted if year of study is 1 or 2. Print specific status flags.

Question 22 – Employee Enterprise Promotion Logic

Build an HR evaluation system centered on an `Employee` class tracking `name`, `department`, `current_position`, `years_worked`, `performance_score`, and `salary`.

- Determine eligibility states without using iterative code structures: If performance score ≥ 90 and tenure ≥ 5 years $\rightarrow$ "Senior Promotion" (20% bump). If performance ≥ 80 and tenure ≥ 3 years $\rightarrow$ "Standard Promotion" (12% bump). If performance $\geq 70 \rightarrow$ "Mandatory Training Required" (5% adjustment). Otherwise, non-eligible. Output the complete comparative ledger.

Question 23 – Emergency Triage Admission Controller

Design a clinical prioritization engine called `Patient` passing `name`, `age`, `temperature`, `blood_pressure_status` (String: "Normal", "High", "Low"), and `oxygen_level`.

- Determine urgency routing: "Emergency" if oxygen level falls below 90% or temperature exceeds 39°C; "Urgent" if blood pressure status is abnormal or age is above 65 years; otherwise "Normal" triage status. Direct patients directly to ICU, Observation Rooms, or General Wards via logic statements.

Question 24 – Cinema Ticket Multi-Tiered Booking

Create an entertainment transactional class called `MovieTicket` accepting `customer_name`, `movie_name`, `age`, `ticket_type` ("Child", "Adult", "VIP"), and `number_of_tickets`.

- Establish price profiles via internal structural constants (e.g., Child=KSh 400, Adult=KSh 800, VIP=KSh 1500). Calculate the absolute gross fee. Apply a flat 10% volume corporate discount if the quantity bought exceeds 4 tickets.

Question 25 – Supermarket Fiscal Point-of-Sale Receipt Engine

Design a retail class called `Purchase` that takes `customer_name`, `item_name`, `quantity`, and `unit_price`. The system must automatically process tax structures and discounts.

- Calculate raw subtotal. Deduct a 5% promotional discount if subtotal exceeds KSh 3,000. Apply standard 16% Value Added Tax (VAT) to the post-discount subtotal. Output a complete, professional invoice receipt showing all calculations lined up properly.

Question 26 – Laptop Technical Recommendation Engine

Create a computer hardware analyzer class called `Laptop` capturing `brand`, `ram_gb`, `storage_gb`, `processor`, and `price`.

- Incorporate an instance evaluation method to derive performance tiers based on memory metrics (e.g., $\text{RAM} \geq 16\text{GB} \rightarrow \text{High-end}$).
- Incorporate a static method `good_for_programming(ram, storage)` that returns `True` only if RAM is at least 8GB and storage space matches or exceeds 256GB.

Question 27 – Municipal Electricity Connection Matrix

Design an infrastructure registration tracking database class called `Customer` mapping `name`, `id_number`, `monthly_income`, and `house_type` ("Permanent", "Semi-Permanent", "Informal").

- Determine deployment criteria: assign low-income households living in informal housing conditions to a highly subsidized basic utility connection program automatically. Map higher-income households to alternative premium service tiers.

Question 28 – Immigration Passport Application Evaluator

Design an automated screening platform via a `PassportApplicant` entity taking `name`, `age`, `nationality`, `passport_type`, and `fee_paid` status (Boolean).

- Develop verification methods returning: "Approved" if the applicant is of valid national origin, age is above 18, and payment status is true; "Pending Documentation" if payment is true but age requires secondary signatures; and "Rejected" for non-payment or administrative failure.

Question 29 – Smart Home Energy Telemetry Device

Create an IoT device class named `SmartAppliance` capturing `device_name`, `power_rating_watts`, and `hours_used_per_day`.

- Automatically compute daily kilowatt-hour (kWh) metrics: $(\text{Watts} * \text{Hours}) / 1000$. Extrapolate to monthly consumption profiles.
- Classify energy load signatures: "Low Power" (Daily consumption below 1 kWh), "Medium Power" (1 to 5 kWh), or "High Power Drain" (Above 5 kWh). Print an efficiency scorecard.

Question 30 – Enterprise Human Resource Master Engine

Create a master class titled `Employee` to manage an enterprise database. The initialization arguments are: `first_name`, `last_name`, `age`, `department`, `position`, `years_worked`, `performance_score`, `basic_salary`, `allowances`, and `overtime_pay`.

- The constructor must automatically generate: `full_name`, `email` (formatted as `first.last@enterprise.co.ke`), `gross_salary`, `tax_deduction` (calculated at 30% of gross), `statutory_deductions` (NHIF fixed at 2.5%, NSSF at 6%, Housing Levy at 1.5%), `total_deductions`, and final net take-home salary.
- Implement instance methods to return an enterprise classification tier (Junior, Mid-Level, Senior, Executive) and promotion candidacy flags based on years worked combined with performance score.
- Incorporate a class variable tracking corporate branding details, a class method to update compensation raise scalars, and a static method checking age parameters against national retirement rules ($\text{age} \geq 60$). Instantiate multiple distinct employee profiles to verify the system across separate data branches.

Challenge Constraints & Methodology

- **Strict Sandbox:** No use of `for` or `while` loops. Iteration must be avoided. Use linear object mapping and direct indexing.
- **No External Modules:** Standard mathematical syntax and basic internal types only. Do not import `math`, `sys`, or other utilities.
- **Self-Directed Validation:** Trace data logic step-by-step. Focus your execution flow on maximizing structural robustness through clean object encapsulation.